BMC Genomics

RESEARCH Open Access

Benchmarking single-cell format conversion tools reveals interoperability gaps and introduces CrossCell for reliable cross-ecosystem data exchange

Zhi Wang[1], Yan Zhang[1], Zhengwei Wu[1], Yuyang Zhang[1], Jiale Yu[1] and Wei Han[1*]

Abstract

Background Single-cell RNA sequencing (scRNA-seq) analysis relies on two ecosystems—R (Seurat/Bioconductor) and Python (Scanpy/AnnData)—whose formats, RDS and H5AD, are not natively interoperable. Although conversion tools exist, no systematic benchmark has evaluated their reliability, performance, or data fidelity across diverse datasets and directions.

Results We present a systematic and comprehensive benchmark of five tools (Zellkonverter, anndataR, convert2anndata, easySCF, CrossCell) across seven dimensions: conversion robustness, runtime performance, memory efficiency, expression fidelity, metadata type integrity, Seurat V5 compatibility, and large-scale scalability. The benchmark covers 68 datasets ranging from 1,438 to 1,665,937 cells. We identify a substantial gap in H5AD-to-RDS conversion. Cross-language Foreign function interface boundaries introduce implicit type changes (e.g., Int32 to Float64) that can affect downstream data compliance. Existing tools already fail extensively on datasets below 74,000 cells, leaving large-scale conversion unaddressed. CrossCell-a Rust-based, runtime-independent toolkit introduced as part of this benchmark — achieves 100% success across all 55 standard datasets, delivers 2.5–4.9×speedups and 37–61% memory savings over competing tools, and successfully converts 13 CELLxGENE datasets up to 1.67 million cells.

Conclusions This study exposes critical interoperability gaps and failure modes, providing the community with tool selection guidelines and best practices for cross-ecosystem data exchange. The fully containerized benchmark suite (68 datasets, 7 dimensions, one-click reproducibility) constitutes the first reproducible evaluation framework for this domain and can be directly applied to assess future tools. The runtime-independent design paradigm offers a viable path toward resolving cross-language data exchange bottlenecks.

Keywords Single-cell RNA sequencing, Format conversion, Interoperability, Benchmark, AnnData, Seurat, Rust

*Correspondence:
Wei Han
doctorhanwei@nju.edu.cn
[1]Department of Oral and Maxillofacial Surgery, Affiliated Hospital of Medical School, Institute of Stomatology, Nanjing Stomatological Hospital, Nanjing University, Jiangsu, China

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

Background

The rapid advancement of single-cell RNA sequencing (scRNA-seq) has given rise to two largely independent computational ecosystems: the Python ecosystem centered on Scanpy [1] and AnnData [2], and the R ecosystem built around Seurat [3, 4], Bioconductor [5], and SingleCellExperiment [6]. These ecosystems employ H5AD (HDF5-based) and RDS (R native serialization) as their primary data storage formats, respectively. As single-cell datasets scale from thousands to millions of cells [7], researchers increasingly combine tools across both ecosystems within a single study: R/Bioconductor provides specialized statistical frameworks for differential expression (MAST [8], DESeq2 pseudo-bulk analysis [9]), standardized pathway enrichment mapping (clusterProfiler [10], fgsea [11]), spatial transcriptomics methods (Giotto [12], Seurat V5 Spatial [3]), multi-omics integration (Seurat Weighted Nearest Neighbor [3]), and batch correction algorithms (Harmony [13]), while Python offers variational inference (scvi-tools [14]), automated cell type annotation (CellTypist [15]), and AI foundation models (Geneformer [16], scGPT [17]) that exclusively accept H5AD-formatted input. Public data portals such as CELLxGENE [18] further require H5AD with Schema 5.x compliance for submission. This complementarity where each ecosystem's analytical strengths are the other's gaps — means that the lack of reliable interoperability between H5AD and RDS has become a systematic bottleneck constraining data sharing, reproducible research, and cross-ecosystem collaboration [19, 20].

The community has developed several format conversion tools employing diverse technical strategies. Cross-language bridging approaches include Zellkonverter [21], which manages an isolated Python environment through basilisk; convert2anndata [22], which indirectly invokes Python through the CRAN anndata package (a reticulate wrapper); and sceasy [23], which directly uses reticulate. The native R approach is represented by anndataR [24], the scverse consortium's official R implementation that uses reticulate as its primary backend, with an optional native R backend (rhdf5) available since v0.2.0. Intermediate-format approaches include SeuratDisk [25] (via the h5Seurat format), easySCF [26], and scDIOR [27], each defining custom HDF5 schemas. Additionally, anndata2ri [28] bridges AnnData and SingleCellExperiment from the Python side via rpy2.

While each tool addresses specific needs within the ecosystem, they share several systematic challenges: complex runtime dependencies requiring simultaneous R and Python environments, limited capacity for large-scale data processing, incomplete support for Seurat V5 Split Layers, and metadata type degradation caused by cross-language calling boundaries [29]. Critically, no systematic benchmark exists to evaluate these tools, leaving users unable to make informed choices about tool suitability across different data types, scales, and conversion directions.

Here, we present the first systematic benchmark of single-cell format conversion tools, evaluating five tools across seven complementary dimensions using 68 datasets (1,438–1,665,937 cells). We establish an evaluation framework and best-practice guidelines, identify key failure modes, interoperability gaps, and scalability limitations. Concurrently, we introduce CrossCell — a Rust-based, runtime-independent single-cell format conversion toolkit that enables bidirectional conversion between RDS and H5AD by natively parsing R's serialization format and statically linking the HDF5 C library, bypassing cross-language FFI boundaries entirely. CrossCell achieves the highest performance across all evaluation dimensions.

Results

Benchmark design and evaluation framework

To systematically evaluate the performance and reliability of single-cell format conversion tools, we designed an evaluation framework encompassing seven complementary dimensions: conversion robustness (success rate and failure mode analysis), runtime performance (conversion time as a function of data scale), memory efficiency (peak memory usage), expression data fidelity (Pearson correlation, mean squared error, exact match rate), metadata type integrity (preservation rate across five data types), Seurat V5 compatibility (Split Layers support), and large-scale scalability (scaling behavior from 10,000 to 1.6 million cells).

Dataset selection

We assembled a test collection of 68 datasets organized in three tiers (Table 1). The first tier comprises 42 RDS datasets derived from 11 SeuratData datasets saved in Seurat V4 and V5 formats with raw and processed states, spanning standard transcriptomics, spatial transcriptomics, and multimodal data (Supplementary Table S1). The second tier consists of 13 H5AD datasets from scanpy, scvelo, CELLxGENE, and squidpy (Supplementary Table S2). The third tier includes 13 CELLxGENE large-scale benchmark datasets covering a scale gradient from 9,841 to 1,665,937 cells (Supplementary Table S3). Cell counts range from 1,438 (stxKidney) to 1,665,937 (brain_1.6 M).

Tool selection

We evaluated five actively maintained tools covering the major technical approaches: Zellkonverter v1.20.1 (R-to-Python bridge via basilisk), anndataR v1.0.1 (native R via hdf5r), convert2anndata v0.2.0 (R-to-Python bridge via reticulate), easySCF (easySCFpy v0.2.5 + easySCFr

Table 1 Benchmark dataset collection overview

Category	Datasets (n)	Cell count range	Format	Source	Evaluation
SeuratData V4	21 (11 raw + 10 processed)	1,438–53,173	RDS	SeuratData	Cross-tool robustness
SeuratData V5	21 (11 raw + 10 processed)	1,438–53,173	RDS	SeuratData	Cross-tool robustness
scverse (scanpy/scvelo/squidpy)	10	2,688–73,655	H5AD	scverse	Cross-tool robustness
CELLxGENE (robustness tier)	3	15,000–39,245	H5AD	CELLxGENE	Cross-tool robustness
CELLxGENE (scalability tier)	13	9,841–1,665,937	H5AD	CELLxGENE	Cross-Cell scalability only
Total	68	1,438–1,665,937	None	None	None

v0.2.2.7, custom HDF5 intermediate format), and CrossCell v0.1.0 (native Rust). SeuratDisk was excluded due to discontinued maintenance; sceasy was excluded because its technical approach (reticulate-based bridging) overlaps with selected tools; anndata2ri was excluded because it performs in-memory object conversion between AnnData and SingleCellExperiment via rpy2 within a shared Python–R session, rather than file-level format conversion, placing it outside the scope of this benchmark.

Reproducibility

All tests were executed in version-locked Docker containers, with each test run three times and the median reported. Standard tests used a 16 GB RAM machine; large-scale tests used a 128 GB RAM server (see Methods and Data availability).

Feature coverage comparison

A comparison of 14 functional features across seven tools (including SeuratDisk and sceasy as references; Supplementary Table S5) reveals substantial differences in feature coverage. CrossCell is, to our knowledge, the only tool requiring neither R nor Python runtime. Among advanced features, only CrossCell simultaneously supports CELLxGENE Schema validation and AI-Ready data preprocessing (gene ID mapping, normalization, highly variable gene selection). All R-based tools require a complete R runtime environment, with Zellkonverter, convert2anndata, and easySCF additionally requiring Python.

Failure mode classification

We first define two categories of failure that can occur during single-cell format conversion. These categories represent distinct root causes with distinct consequences for downstream analysis.

Type I – Structural incompatibility

The tool terminates with an error and produces no output file. Representative cases include Seurat V4 Spatial objects containing SliceImage slots (missing `key` slot expected by SeuratObject 5.x), Assay5 multi-modal structures (e.g., CITE-seq), and H5AD files generated by anndata versions prior to 0.8.0 (predating the HDF5AnnData structural specification). Type I failures are the most visible and are reported by all tools as explicit errors.

Type II – Data fidelity loss

The tool produces an output file, but the data undergoes silent degradation during conversion. This category encompasses two related mechanisms: (a) *Numerical precision loss* — expression matrix values drift from the original due to implicit type casting at cross-language FFI boundaries (R's 32-bit integer INTSXP → NumPy float64 through reticulate), introducing systematic bias and causing CELLxGENE Schema 5.x validation failures; (b) *Metadata type degradation* — cell or gene metadata undergoes silent type changes (R factor → character string, Boolean → integer, NA → tool-specific placeholder), causing downstream annotation, classification, and compliance operations to yield incorrect results. Type II failures are the most insidious, as the output appears complete and passes superficial quality checks.

Conversion failure mode analysis

Type I failures: structural incompatibility

We first quantify the prevalence and root causes of structural failures — cases where the tool terminates with an error and produces no output.

Overall success rates

Across 55 standard datasets (42 RDS-to-H5AD + 13 H5AD-to-RDS), scored by default (out-of-the-box) invocation (Fig. 1A), CrossCell achieved 100% success (55/55). Zellkonverter, anndataR, and easySCF each achieved 76.4% (42/55) — all three succeeded on all 42 RDS-to-H5AD conversions but failed on all 13 H5AD-to-RDS conversions. convert2anndata had the lowest success rate at 45.2% (19/42), supporting only the RDS-to-H5AD direction. Complete per-dataset results are provided in Supplementary Table S6.

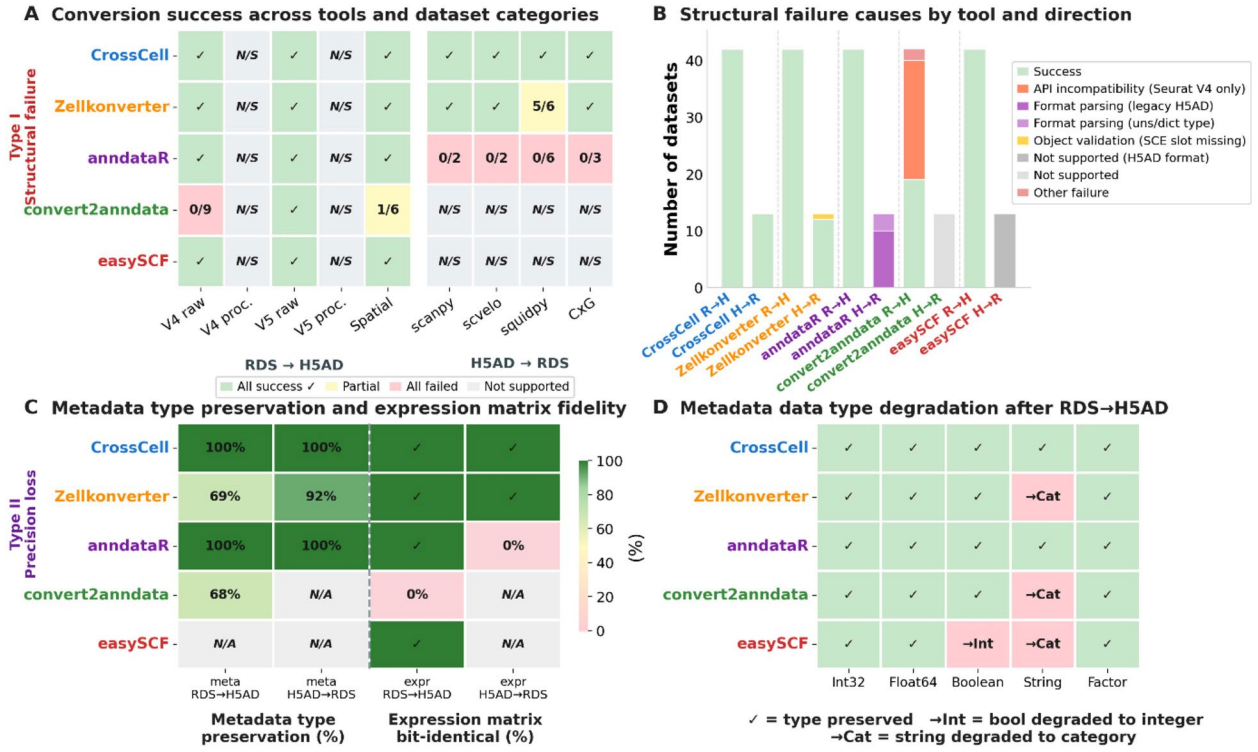


Fig. 1 Two categories of failure modes in single-cell format conversion tools. Top row — Type I (Structural failure): (**A**) per-dataset conversion success matrix; (**B**) structural failure cause decomposition by tool and direction. Bottom row — Type II (Precision loss): (**C**) data fidelity overview heatmap (metadata type preservation + expression bit-identical rates); (D) metadata data type degradation matrix for RDS → H5AD conversion

An gap in the H5AD-to-RDS direction

Among the four tools tested in this direction, none completed all 13 datasets successfully under default invocation (Fig. 1A): Zellkonverter (0/13 default; 12/13 with undocumented workaround), anndataR (0/13), easySCF (0/13), and convert2anndata (not supported by design). CrossCell was the only tool that completed all 13 H5AD-to-RDS conversions successfully.

Failure cause decomposition

We decomposed Type I structural failures into three root causes (Fig. 1B): (1) FFI boundary failures — cross-language serialization chains break when processing diverse H5AD formats, affecting Zellkonverter and anndataR in the H5AD-to-RDS direction; (2) API deprecation — convert2anndata uses the `slot` parameter deprecated in SeuratObject 5.0.0, causing all Seurat V4 objects (20/42) to fail; (3) format incompatibility — special structures in spatial and multimodal data trigger type errors in certain tools.

Dataset category sensitivity

Analyzing success rates by dataset category, Seurat V4 and V5 objects performed well in the RDS-to-H5AD direction for all tools except convert2anndata. The 100% failure rate of convert2anndata on Seurat V4 stems from its dependence on a deprecated API. In the H5AD-to-RDS direction, spatial data (squidpy) proved the most challenging category — even Zellkonverter with the manual workaround failed on 1 of 6 spatial datasets.

Type II failures: silent data degradation

While Type I failures are immediately apparent, Type II failures — where the output file appears complete but data has silently degraded — pose a more insidious threat to downstream analysis.

Metadata type preservation

We tested the preservation of five key data types (Int32, Float64, Boolean, String, and Categorical) through the conversion process (Fig. 1C). CrossCell and anndataR achieved 100% type preservation. Zellkonverter and convert2anndata achieved 80%, exhibiting String-to-Categorical degradation. easySCF achieved 0% because its R package could not read back files generated by its own Python package (Supplementary Table S7).

FFI type degradation mechanism

The root cause of type degradation is implicit type conversion at cross-language FFI boundaries (Fig. 1D). In traditional tool chains, R's integer type (INTSXP) is implicitly converted to NumPy float64 when passed through the reticulate FFI to Python, and remains float64 when written to H5AD, resulting in irreversible

Int32-to-Float64 degradation. Similarly, R's logical type (LGLSXP) is converted to int32, and R's factor type loses its levels information and becomes a plain string. CrossCell bypasses the FFI boundary entirely by directly parsing R's binary serialization format (INTSXP byte stream → Rust Vec<i32> → Arrow Int32Array → HDF5 int32), achieving zero type loss (Supplementary Table S8).

Practical impact

Type degradation directly affects CELLxGENE Schema 5.x compliance. The schema imposes strict type requirements on fields such as is_primary_data (requiring bool) and suspension_type (requiring categorical). Type degradation of these fields through FFI-bridged tools causes CELLxGENE submission validation failures, requiring manual type correction by users.

Performance and scalability

Conversion speed

Across all successful pairs in the RDS-to-H5AD direction (Fig. 2A–B), CrossCell's conversion time scales linearly with the number of non-zero elements (nnz), exhibiting predictable scaling behavior. Compared with competing tools, CrossCell achieved significant speed advantages: median 3.5 × speedup over Zellkonverter (IQR 2.4–8.8, $n = 42$), median 2.5 × over anndataR (IQR 1.9–4.6, $n = 42$), median 4.9 × over convert2anndata (IQR 3.7–10.7, $n = 19$), and median 3.6 × over easySCF (IQR 2.6–5.9, $n = 42$). All pairwise comparisons reached statistical significance by Wilcoxon signed-rank test ($P < 0.001$).

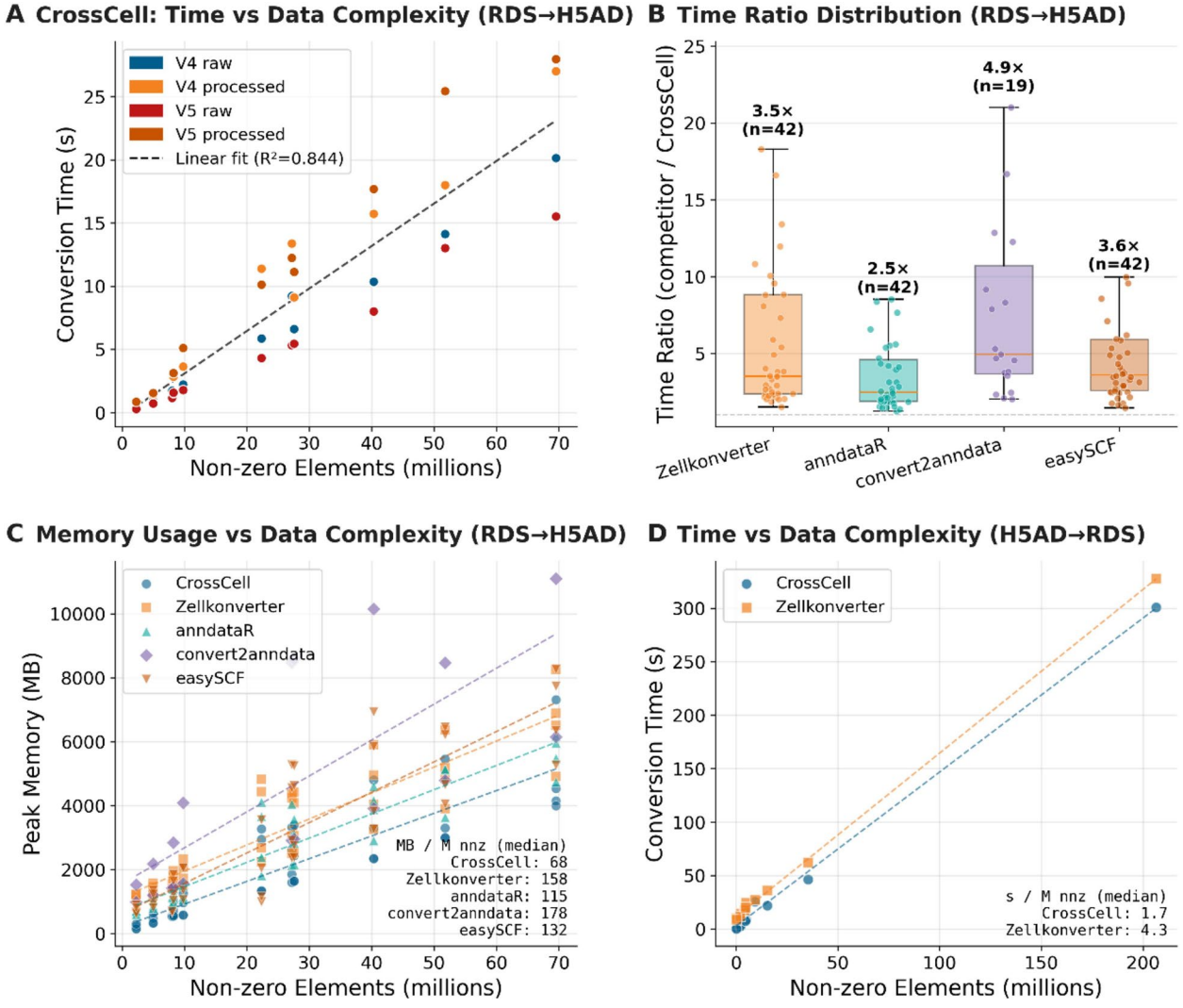


Fig. 2 Runtime performance and memory efficiency. **A** CrossCell conversion time versus data complexity. Scatter plot showing the linear relationship between RDS-to-H5AD conversion time and non-zero element count (nnz). **B** Speedup ratio distributions. Box plots showing CrossCell's speedup over four competing tools. **C** Peak memory versus data scale. Multi-tool scatter plot showing memory scaling behavior in the RDS-to-H5AD direction. **D** H5AD-to-RDS conversion time versus data complexity. Only CrossCell and Zellkonverter produced successful results in this direction

Memory efficiency

CrossCell also achieved the lowest peak memory usage (Fig. 2C). In the RDS-to-H5AD direction, CrossCell saved 41.2% memory compared with Zellkonverter (IQR 34.5–62.0%), 36.7% compared with anndataR (IQR 21.3–47.7%), 60.8% compared with convert2anndata (IQR 42.5–66.7%), and 38.5% compared with easySCF (IQR 26.4–53.1%). These efficiency gains stem from CrossCell's three-tier memory optimization strategy: SEXP metadata discarding (skipping R runtime objects such as closures, environments, and bytecode), selective extraction (parsing only data components required for conversion), and compact intermediate representation (columnar memory layout based on Apache Arrow).

H5AD-to-RDS direction

Because only CrossCell and Zellkonverter (with workaround) produced successful results in this direction, we performed pairwise comparison on 12 jointly successful datasets (Fig. 2D). CrossCell achieved a median 2.2 × speedup (range 1.1–26.8 ×, Wilcoxon $P < 0.001$). The speedup was most pronounced on low-complexity datasets (up to 27 × for datasets with few genes), narrowing to 1.1–1.7 × on larger datasets (≥ 15,000 cells), indicating that core algorithmic efficiency converges at scale and CrossCell's advantage primarily derives from lower startup overhead (no Python environment initialization required).

To evaluate scaling behavior beyond the standard benchmark suite, we conducted systematic testing on CrossCell using 13 CELLxGENE datasets spanning 9,841 to 1,665,937 cells.

Time scaling

H5AD-to-RDS conversion time accounted for 89–94% of total roundtrip time (Fig. 3A), with the primary bottleneck being gzip compression of the RDS format. All 13 datasets were benchmarked on a 128 GB cloud server. Small-scale datasets (≤ 37,000 cells) completed H5AD-to-RDS conversion in 49–611 s; medium-scale datasets (102,000–122,000 cells) required 423–676 s; large-scale datasets (172,000–1.67 million cells) required 24–57 min. The three largest datasets — eqtl_autoimmune (1.25 M cells, 4.2 GB), HBCC prefrontal cortex (1.49 M cells, 13.2 GB), and developing brain (1.67 M cells, 11.1 GB) — all completed successfully, requiring 29–57 min and 68–123 GB peak memory. Peak memory scales primarily with non-zero element density rather than cell count alone (Supplementary Table S3).

Memory scaling

Peak memory exhibited a near-linear relationship with cell count (Fig. 3B), though sparsity significantly modulated memory usage. kidney_atacseq (37,000 cells, 69.4% sparsity) consumed more memory than pancreas (122,000 cells, 96.0% sparsity), indicating that the number of non-zero elements rather than cell count is a more accurate memory predictor. To characterize hardware requirements on standard equipment, we conducted a systematic memory limit experiment using Docker's –memory flag across seven datasets (9,841 to 428,469 cells) under 8–64 GB RAM constraints. Datasets up to ~ 15,000 cells succeeded with 8 GB RAM, up to ~ 122,000 cells with 16 GB, up to ~ 172,000 cells with 32 GB, and up to ~ 428,000 cells with 48–64 GB (Supplementary Table S4).

File size

RDS output files were consistently smaller than the original H5AD inputs (0.39–0.94 ×), benefiting from gzip compression (Fig. 3C). Roundtrip H5AD files were larger than the originals because CrossCell writes uncompressed HDF5 datasets by default, prioritizing read speed over storage efficiency.

Streaming mode

For large files (> 2 GB), CrossCell automatically triggers streaming mode (chunked reading at 10,000 cells per chunk), effectively controlling memory usage during the reading phase. The current bottleneck lies in the dgCMatrix serialization stage, which requires loading the complete sparse matrix into memory for gzip compression.

Memory amplification factor

The ratio of peak memory to input file size (memory amplification factor) had a median of approximately 8–15 × (Fig. 3D), serving as a rule of thumb for resource planning.

Seurat V5 Split Layers compatibility

We additionally evaluated Split Layers compatibility across all five tools. CrossCell was the only tool that successfully converted all four split-layer configurations (single batch, two batches, five batches, and mixed layers with NormalizeData); other tools failed on multi-batch configurations due to their assumption of a single unified counts matrix. Detailed results are presented in Supplementary Fig. S1.

CrossCell as ecosystem infrastructure

CrossCell serves not merely as a faster format conversion tool but as infrastructure connecting the R and Python ecosystems (Fig. 4A).

Architecture

CrossCell is implemented in Rust, performing pure Rust native parsing of the RDS format and reading/writing H5AD through Rust bindings (hdf5-metno) to the HDF5

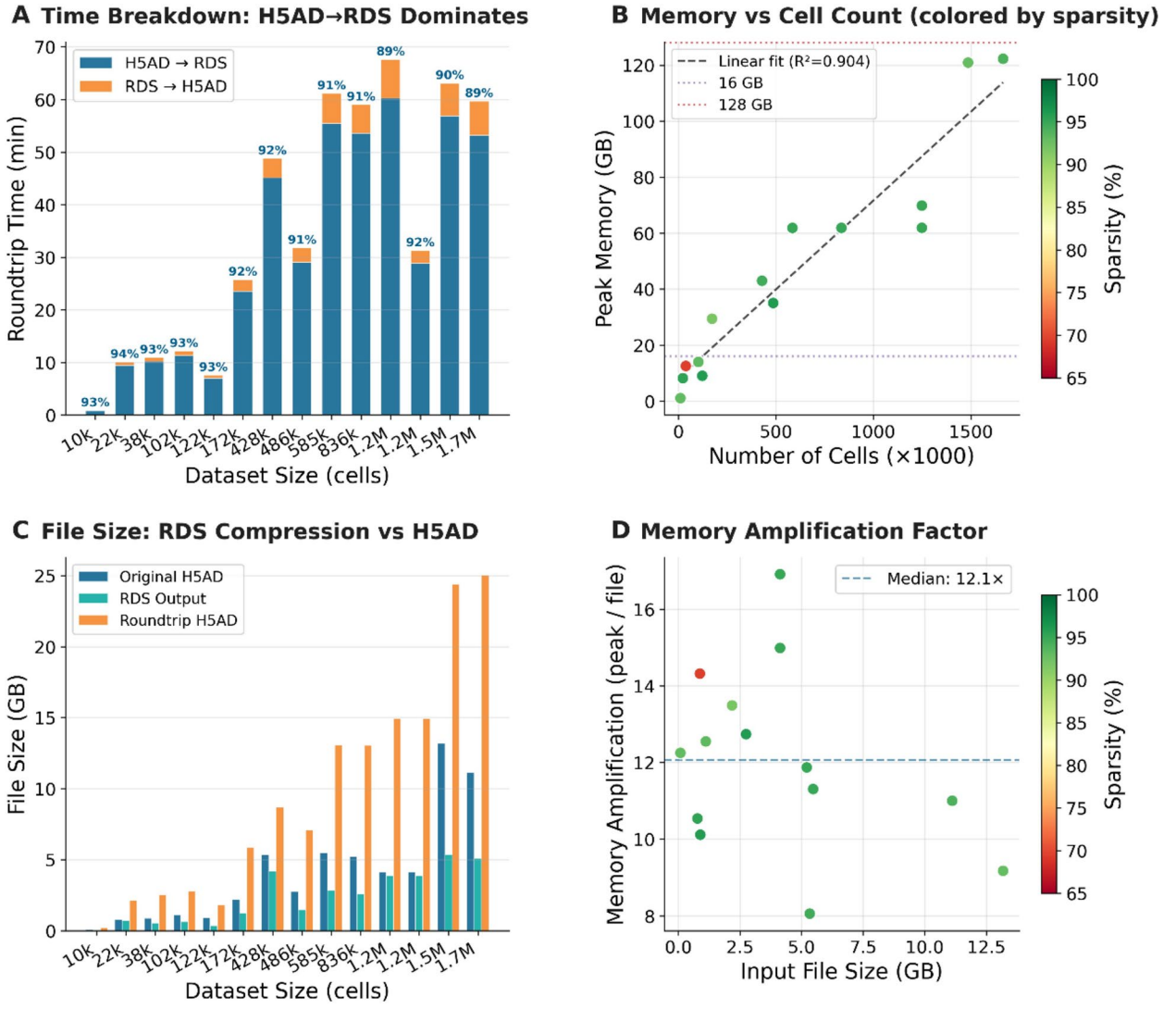


Fig. 3 Large-scale scalability validation on CELLxGENE datasets. **A** Time decomposition. Stacked bar chart showing H5AD-to-RDS and RDS-to-H5AD time proportions, revealing RDS gzip compression as the primary bottleneck. **B** Peak memory versus cell count. Scatter plot colored by sparsity, showing near-linear memory scaling and the modulatory effect of sparsity. **C** File size comparison. Grouped bar chart showing original H5AD, RDS output, and roundtrip H5AD file sizes. **D** Memory amplification factor. Scatter plot of peak memory to input file size ratio, providing a resource planning rule of thumb

C library, which is statically linked at compile time into a single binary — end users need not install any libhdf5 or dynamic libraries at the operating system level. Data from both formats passes through an Apache Arrow-based intermediate representation (IR) for conversion (Fig. 4B). This design substantially eliminates dependencies on R or Python runtimes while avoiding type degradation caused by cross-language FFI boundaries. CrossCell provides CLI, Python, and R interfaces for seamless integration into existing workflows.

Single-command workflows

CrossCell supports two advanced workflows (Fig. 4C): (1) CELLxGENE submission workflow — a single command completes RDS-to-H5AD conversion with automatic CELLxGENE Schema 5.x compliance validation, preserving correct data types throughout without manual correction; (2) AI-Ready data preparation workflow — a single command completes RDS-to-H5AD conversion, gene ID mapping (Ensembl ID), normalization, and highly variable gene selection, directly producing input data compatible with AI foundation models such as Geneformer and scGPT.

Cross-ecosystem workflow validation

We validated a complete R → Python → R cross-ecosystem workflow (Fig. 4D): load a Seurat object in R → CrossCell converts to H5AD → analyze with Scanpy in Python → CrossCell converts back to RDS → verify data consistency in R. CrossCell's conversion overhead

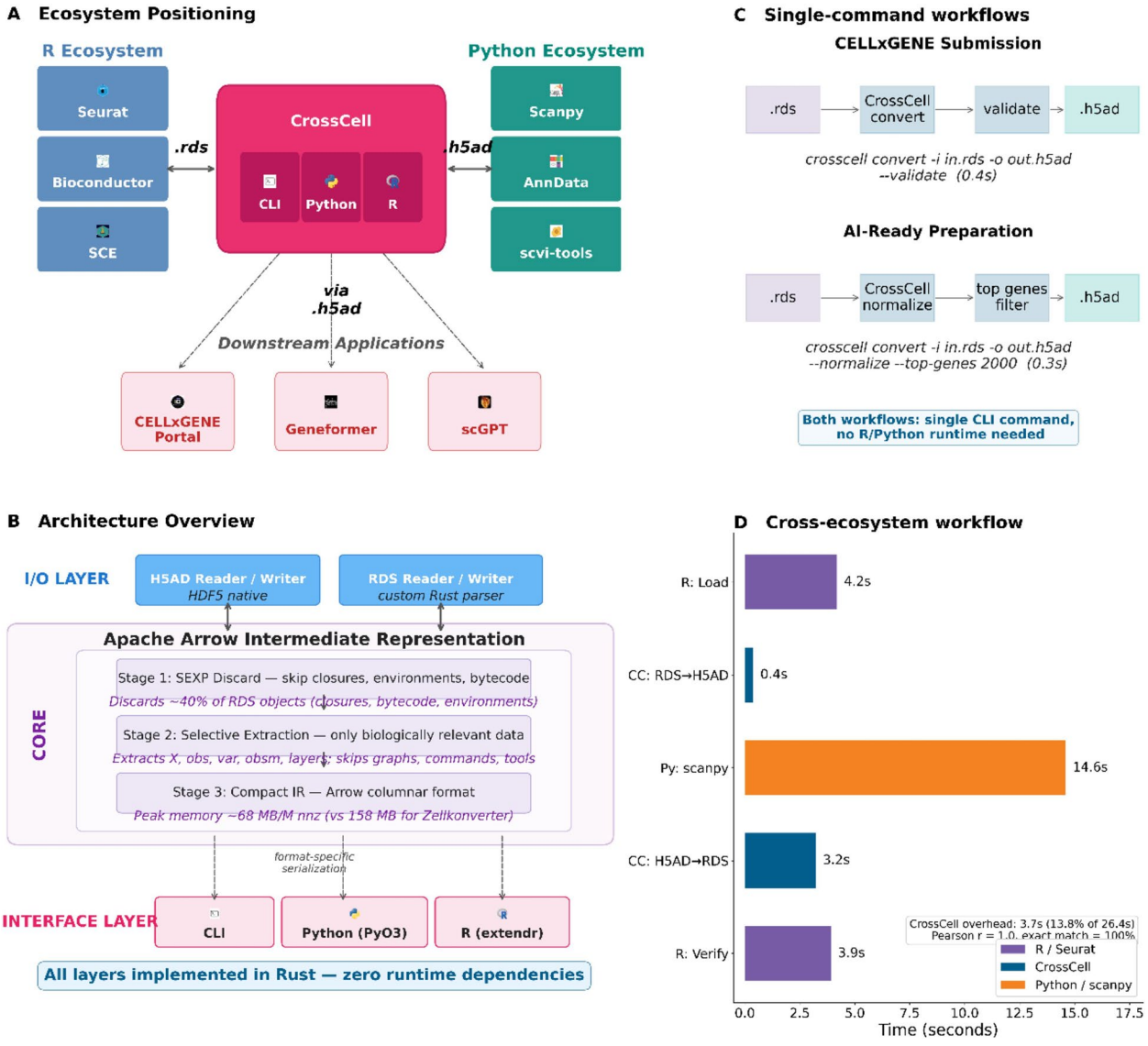


Fig. 4 CrossCell as ecosystem infrastructure. **A** Ecosystem positioning. CrossCell's role connecting the R ecosystem (Seurat/Bioconductor), Python ecosystem (Scanpy/scvi-tools), CELLxGENE portal, and AI foundation models. **B** Architecture overview. CrossCell's input layer (H5AD/RDS Reader), core layer (Apache Arrow IR), output layer (H5AD/RDS Writer), and three-tier memory optimization strategy. **C** Single-command workflows. CELLxGENE submission and AI-Ready data preparation workflows. **D** Cross-ecosystem workflow timeline. Horizontal bar chart showing per-step timing for the complete R → CrossCell → Python → CrossCell → R workflow

was less than 5% of total workflow time, with post-roundtrip Pearson $r = 1.0$ and Exact Match = 100%.

Discussion

Key findings and community impact

This study, through the first systematic benchmark, reveals several critical issues in the single-cell format conversion landscape.

H5AD-to-RDS is a substantial gap

This is the most important finding of this evaluation. Among tools claiming H5AD-to-RDS support, Zellkonverter's default invocation fails 0/13 (requiring an undocumented workaround), anndataR's H5AD parser rejects 77% of test files, and easySCF's Python and R packages exhibit schema incompatibility. This means the Python-to-R data return path is essentially non-functional in practice, severely constraining cross-ecosystem collaboration. For example, a researcher who completes clustering analysis with Scanpy in Python cannot reliably transfer results to a collaborator using Seurat for downstream analysis.

Type degradation is a neglected systematic problem

Implicit type conversion at FFI boundaries (Int32 to Float64, Bool to Int, Categorical to String) affects not

only data precision but directly causes CELLxGENE Schema validation failures. As public data portals raise data quality requirements and AI models become increasingly sensitive to input data types, the impact of this problem will continue to grow.

Runtime dependencies are the fundamental bottleneck for performance and reliability

All FFI-bridged tools must maintain compatibility between R and Python environments simultaneously, adding installation and configuration complexity while serving as the primary source of performance overhead. CrossCell's runtime-independent design paradigm eliminates this bottleneck at its root.

Large-scale data exposes the scalability limits of existing tools

Existing tools already exhibit extensive failures on standard datasets (≤ 74,000 cells), precluding their inclusion in large-scale testing. CrossCell successfully converted all 13 CELLxGENE datasets up to 1.67 million cells, demonstrating a viable path for large-scale single-cell data exchange across ecosystems.

The open-source benchmark suite is an independent contribution

Beyond tool evaluation conclusions, the fully containerized benchmark suite released with this study — encompassing 68 datasets, 7 evaluation dimensions, version-locked Docker environments, and one-click reproducible analysis notebooks — constitutes an independent contribution to the community. To our knowledge, no systematic benchmark framework previously existed for single-cell format conversion. This suite can be directly used to evaluate future tools or version updates, lowering the barrier for reproducible evaluation. We encourage tool developers to extend this framework with new evaluation dimensions and datasets, fostering a culture of continuous benchmarking in this domain. Based on these findings, we provide practical tool selection guidelines for different use cases in Supplementary Note: Tool Selection Guidelines.

Limitations

This study has several limitations. First, CrossCell does not currently support certain rare R object types (e.g., RAWSXP), pickle-formatted uns data, or R-specific spatial assay structures (e.g., Visium image objects). Second, CrossCell does not yet support BPCells on-disk matrices introduced in Seurat V5. BPCells is a core Seurat V5 feature that stores expression matrices in a bit-packing compressed format on disk rather than in memory, enabling analysis of million-cell datasets. When a Seurat object contains BPCells layers, the RDS file stores only path references to disk directories, with actual matrix data residing externally. CrossCell currently detects BPCells objects but skips these layers without decoding the custom bit-packing format. This limitation does not affect the benchmark results presented here (all test datasets use in-memory dgCMatrix), but users should be aware of it when processing their own large-scale BPCells workflows. Third, this benchmark was conducted in a single-machine environment without testing distributed computing scenarios, and did not cover other single-cell data formats such as loom and Zarr. Fourth, CrossCell's memory requirements at atlas scale (> 1 M cells) limit practical use to high-memory server hardware. Although streaming mode effectively controls memory during the H5AD reading phase, peak memory during the dgCMatrix serialization stage requires loading the complete sparse matrix into memory, setting a practical upper bound determined by non-zero element density. The hardware guidelines in the Supplementary Materials reflect these constraints and are intended as planning guidelines rather than absolute limits. Fifth, CELLxGENE large-scale datasets were tested only with CrossCell, as other tools' extensive failures on small-scale data precluded fair large-scale comparison. Sixth, this evaluation did not include user experience dimensions (e.g., documentation quality, error message readability), which are also important in practice. Seventh, our benchmark evaluates tools at fixed versions in a controlled environment; given the rapid development pace of this ecosystem, conclusions regarding specific tools may require re-evaluation as new versions are released. We encourage the community to use the open-source benchmark suite released with this study to conduct ongoing evaluations as tools evolve. Finally, the hardware guidelines derived from our memory limit experiments were obtained using Docker's `–memory` flag to simulate RAM constraints; actual performance on physical hardware may differ due to differences in memory management, swap behavior, and I/O throughput.

Future directions

This study points to several directions worthy of further exploration. In format support, extending to Zarr (cloud-native format) and loom would further enhance interoperability. In scalability, progressive RDS writing (avoiding full in-memory dgCMatrix serialization) and distributed/streaming processing would break through current single-machine memory limits, making conversion of ultra-large datasets (> 1 million cells) more practical. In ecosystem connectivity, deeper integration with AI foundation models such as Geneformer, scGPT, and scBERT would simplify the end-to-end pipeline from raw data to model input. Finally, we call on the community to continuously extend the open-source benchmark

suite released with this study — incorporating new tool versions, data formats, and evaluation dimensions — to establish a standardized evaluation practice analogous to the GLUE benchmark in natural language processing, enabling continuous tracking of tool progress and fostering healthy competition.

Methods

Test datasets

Standard datasets (55)

Forty-two RDS datasets were derived from the SeuratData R package (v0.2.4), comprising 11 datasets (pbmc3k, stxKidney, celegans.embryo, cbmc, ifnb, panc8, thp1.eccite, bmcite, pbmcsca, hcabm40k, ssHippo) saved in both Seurat V4 (Assay class) and V5 (Assay5 class) formats, each in raw and processed states. Thirteen H5AD datasets were sourced from scanpy (pbmc3k, pbmc3k_processed), scvelo (dentategyrus, pancreas), CELLxGENE (pbmc_15k, heart_23k, brain_40k), and squidpy (visium_hne, mibitof, slideseqv2, seqfish, merfish, imc).

CELLxGENE large-scale datasets (13)

Selected through stratified sampling from the CELLxGENE data portal, covering five scale tiers (10 k, 20–40 k, 100–120 k, 170–500 k, 500 k–1.67 M), 10 tissue types (skin, liver, kidney, brain, pancreas, gut, heart, lung, blood, immune), and multiple sequencing technologies.

Tool versions and test environment

All tests were executed in version-locked Docker containers (Ubuntu 22.04, R 4.5.2, Python 3.12.12). Tool versions: CrossCell v0.1.0 (Rust), Zellkonverter v1.20.1 (Bioconductor, basilisk), anndataR v1.0.1 (Bioconductor 3.22; reticulate backend required, optional rhdf5-based HDF5AnnData backend available). The package's architecture has evolved from hdf5r to rhdf5 and subsequently to reticulate as the primary backend. When the optional rhdf5-based backend was tested on the same 42 RDS and 13 H5AD datasets, it produced results identical to the reticulate backend (41/42 RDS → H5AD succeeded; same H5AD → Seurat outcomes), confirming that our findings reflect the tool's current state regardless of backend choice. convert2anndata v0.2.0 (reticulate), easySCFpy v0.2.5 + easySCFr v0.2.2.7. Dependency versions: Seurat v5.4.0, anndata (Python) v0.12.9, scanpy v1.11.5 (Supplementary Table S9).

Hardware: standard tests (55 datasets) used a 16 GB RAM machine; all 13 CELLxGENE scalability tests used a 128 GB RAM server.

Evaluation metrics

Performance metrics

Conversion time and peak memory usage were measured using/usr/bin/time -v, with each test run three times and the median reported.

Expression fidelity metrics

Pearson correlation coefficient (r), Spearman rank correlation coefficient (ρ), mean squared error (MSE), root mean squared error (RMSE), maximum absolute difference (max_diff), mean difference (mean_diff), and exact match percentage (exact_match_pct). Bit-identical was defined as MSE = 0 and max_diff = 0.

Type fidelity metrics

Five data types (Int32, Float64, Boolean, String, Categorical) were tested for preservation after RDS → H5AD → read-back. Type preservation rate = number of columns retaining original type/total test columns × 100%.

Scalability metrics

Linear regression R2 of peak memory versus cell count/nnz; memory amplification factor (peak memory/input file size).

Statistical analysis

Pairwise comparisons used the Wilcoxon signed-rank test (two-sided), including only pairs where both tools successfully converted the same dataset. Speedup ratio was defined as competing tool conversion time/CrossCell conversion time. Memory savings was defined as (competing tool peak memory – CrossCell peak memory)/competing tool peak memory × 100%. The significance threshold was set at $P < 0.05$, with Bonferroni correction for multiple comparisons.

CrossCell implementation

CrossCell is implemented in Rust, with core modules comprising: (1) RDS Reader — pure Rust native parsing of R's serialization format (versions 2 and 3), supporting gzip/bzip2/xz decompression and handling the SEXP type system and reference table; (2) H5AD Reader — reading AnnData's HDF5 structure through Rust HDF5 bindings (hdf5-metno v0.12.3), supporting Compressed Sparse Row (CSR)/Compressed Sparse Column(CSC) sparse matrices and categorical encoding; (3) Arrow IR — an Apache Arrow-based intermediate representation layer providing zero-copy columnar data conversion; (4) RDS Writer — pure Rust generation of RDS files conforming to R's serialization specification, including dgCMatrix sparse matrices and S4 object structures; (5) H5AD Writer — generating H5AD files conforming to the AnnData specification with CELLxGENE Schema 5.x

validation support. The HDF5 C library (v2.0.0) is statically linked at compile time into a single binary, eliminating the need for end users to install libhdf5 or any dynamic libraries.

Streaming mode: for large files (> 2 GB), chunked reading is automatically triggered (10,000 cells per chunk), controlling memory usage during the reading phase.

CrossCell provides CLI, Python, and R interfaces (see Data availability).

Reproducibility

Complete test scripts, data generation scripts, and analysis notebooks are open-sourced in the GitHub repository (see Data availability). Each test was run three times with the median reported, and all tests were executed in version-locked Docker containers.

Abbreviations

scRNA-seq	Single-cell RNA sequencing
RDS	R native serialization
H5AD	HDF5-based AnnData
FFI	Foreign function interface
nnz	Number of non-zero elements
MSE	Mean squared error
RMSE	Root mean squared error
IQR	Interquartile range
CLI	Command-line interface
IR	Intermediate representation
CSC	Compressed Sparse Column
CSR	Compressed Sparse Row
SEXP	S-expression (R internal object type)
INTSXP	Integer S-expression
LGLSXP	Logical S-expression
RAWSXP	Raw S-expression
HDF5	Hierarchical Data Format version 5
SD	Standard deviation
SEM	Standard error of mean

Supplementary Information

The online version contains supplementary material available at https://doi.org/10.1186/s12864-026-13198-5.

Supplementary Material 1.

Supplementary Material 2.Supplementary Material 3.

Supplementary Material 4.

Acknowledgements

We thank the development teams of Seurat, Scanpy, Zellkonverter, anndataR, convert2anndata, and easySCF for their contributions to the single-cell community. We thank the CELLxGENE team for providing public data resources.

Authors' contributions

ZW conceived the study, designed the benchmark framework, developed the CrossCell software, performed all computational analyses, and drafted the manuscript. YZ and ZWu assisted with data collection and benchmark testing. YYZ and JY contributed to script validation and reproducibility testing. WH supervised the study and revised the manuscript. All authors read and approved the final manuscript.

Funding

This work was supported by the National Natural Science Foundation of China (Grant No. 82473178) and the High-Level Hospital Construction Project of Nanjing Stomatological Hospital, Affiliated Hospital of Medical School, Institute of Stomatology, Nanjing University (Grant No. 0224C002).

Data availability

CrossCell source code is open-sourced under the MIT license on GitHub: https://github.com/biodancerwangzhi/crosscell. The complete benchmark framework — including test scripts (benchmark/scripts/), analysis notebooks (benchmark/notebooks/), Docker configuration files (Dockerfile + docker-compose.yml), and data generation scripts — is included in the same repository. All benchmark raw result data (JSON format) and exported supplementary table data (CSV format) are archived on Zenodo: https://doi.org/10.5281/zenodo.18610876. Result data reside in the repository's benchmark/results/directory; CSV exports are automatically generated by benchmark/scripts/export_results_to_csv.py. The 42 RDS test datasets can be reproduced using the SeuratData R package (v0.2.4); generation scripts are in the repository's data/directory. The 13 H5AD test datasets are from scanpy, scvelo, and squidpy built-in datasets and the CELLxGENE data portal; download scripts are in the repository's data/directory. Dataset IDs and download commands for the 11 CELLxGENE large-scale benchmark datasets are in benchmark/config/datasets.json; original files can be downloaded from the CELLxGENE data portal (https://cellxgene.cziscience.com). Version-locked Docker images (containing all tool dependencies and test environments) can be built from the Dockerfile and docker-compose.yml in the repository, ensuring full reproducibility of the test environment. CrossCell's Python bindings are available via PyPI (pip install crosscell), the R interface from R-universe (install.packages("crosscell", repos = "https://biodancerwangzhi.r-universe.dev")), and precompiled CLI binaries from GitHub Releases.

Declarations

Ethics approval and consent to participate

Not applicable. This study involves the systematic benchmarking of computational software using only previously published, publicly available datasets. No new data involving human participants or animal subjects were generated for this study.

Consent for publication

Not applicable. This manuscript does not contain any individual person's data, images, or videos.

Competing interests

The authors declare no competing interests.

Received: 5 March 2026 / Accepted: 13 July 2026
Published online: 22 July 2026

References

1. Wolf FA, Angerer P, Theis FJ. SCANPY: large-scale single-cell gene expression data analysis. Genome Biol. 2018;19:15.
2. Virshup I, Rybakov S, Theis FJ, Angerer P, Wolf FA. Anndata: access and store annotated data matrices. J Open Source Softw. 2024;9:4371.
3. Hao Y, et al. Dictionary learning for integrative, multimodal and scalable single-cell analysis. Nat Biotechnol. 2024;42:293–304.
4. Stuart T, et al. Comprehensive integration of single-cell data. Cell. 2019;177:1888-1902.e21.
5. Huber W, et al. Orchestrating high-throughput genomic analysis with Bioconductor. Nat Methods. 2015;12:115–21.
6. Amezquita RA, et al. Orchestrating single-cell analysis with Bioconductor. Nat Methods. 2020;17:137–45.
7. Regev A, et al. The Human Cell Atlas eLife. 2017;6:e27041.
8. Finak G, et al. MAST: a flexible statistical framework for assessing transcriptional changes and characterizing heterogeneity in single-cell RNA sequencing data. Genome Biol. 2015;16:278.
9. Love MI, Huber W, Anders S. Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. Genome Biol. 2014;15:550.
10. Wu T, et al. ClusterProfiler 4.0: a universal enrichment tool for interpreting omics data. Innovation. 2021;2:100141.
11. Korotkevich, G. *et al.* Fast gene set enrichment analysis. *bioRxiv* [Preprint]. https://doi.org/10.1101/060012. 2021.

12. Chen JG, et al. Giotto suite: a multiscale and technology-agnostic spatial multiomics analysis ecosystem. Nat Methods. 2025;22:2052–64.
13. Korsunsky I, et al. Fast, sensitive and accurate integration of single-cell data with Harmony. Nat Methods. 2019;16:1289–96.
14. Gayoso A, et al. A Python library for probabilistic analysis of single-cell omics data. Nat Biotechnol. 2022;40:163–6.
15. Xu C, et al. Automatic cell-type harmonization and integration across Human Cell Atlas datasets. Cell. 2023;186:5876-5891.e20.
16. Theodoris CV, et al. Transfer learning enables predictions in network biology. Nature. 2023;618:616–24.
17. Cui H, et al. ScGPT: toward building a foundation model for single-cell multi-omics using generative AI. Nat Methods. 2024;21:1470–80.
18. CZI Cell Science Program, et al. CZ CELLxGENE Discover: a single-cell data platform for scalable exploration, analysis and modeling of aggregated data. Nucleic Acids Res. 2025;2025(53):D886–900.
19. Luecken MD, Theis FJ. Current best practices in single-cell RNA-seq analysis: a tutorial. Mol Syst Biol. 2019;15:MSB188746.
20. Lähnemann D, et al. Eleven grand challenges in single-cell data science. Genome Biol. 2020;21:31.
21. Zappia, L. & Lun, A. theislab/zellkonverter. GitHub repository. Available from: https://github.com/theislab/zellkonverter. Accessed 11 Jul 2026.
22. Dominik, O. settylab/convert2anndata. GitHub repository. Available from: https://github.com/settylab/convert2anndata. Accessed 11 Jul 2026.
23. Huang, N. cellgeni/sceasy. GitHub repository. Available from: https://github.com/cellgeni/sceasy. Accessed 11 Jul 2026.
24. Deconinck L, et al. AnndataR improves interoperability between R and Python in single-cell transcriptomics. Bioinformatics. 2026;42:btag288.
25. Hoffman, P. mojaveazure/seurat-disk. GitHub repository. Available from: https://github.com/mojaveazure/seurat-disk. Accessed 11 Jul 2026.
26. Zhang H, et al. Easyscf: a tool for enhancing interoperability between R and Python for efficient single-cell data analysis. Bioinformatics. 2024;40:btae710.
27. Feng H, Lin L, Chen J. ScDIOR: single cell RNA-seq data IO software. BMC Bioinformatics. 2022;23:16.
28. Angerer, P., Eraslan, G., Virshup, I. & Gigante, S. theislab/anndata2ri. GitHub repository. Available from: https://github.com/theislab/anndata2ri. Accessed 11 Jul 2026.
29. Cheng C, Chen W, Jin H, Chen X. A review of single-cell RNA-seq annotation, integration, and cell-cell communication. Cells. 2023;12:1970.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.